NEST 滚动字幕

技术实现 · 单文件参考

面向接手实现或提取组件的 AI 编程代理。目标是在语音消息中实现逐字已读状态、暂停续播、点字定位，以及流式通话的字幕推进，并允许接入方使用自己的界面样式。

整理日期：2026 年 10 月 16 日。本文提供字幕核心算法与可迁移的集成方案；接口路由、限额和缓存命名均由接入方自行定义，不提供 NEST 的线上地址、私有接口映射或部署配置。文中的组件接口是便于迁移的设计约定，不能当成已经发布的 npm API。

这一个文件包含完整的接入说明、五个核心算法函数和合成数据测试，直接交给 AI 编程助手即可，无需其他附件。不包含私人地址、密钥、聊天记录、原应用目录或气泡样式源码。本文提供可复用实现参考；接入方仍需连接自己的播放器、服务端和界面。

1 交互定义

这里的“滚动字幕”指气泡内文字随播放进度逐字变深。字幕保持自然换行；不是横向跑马灯，也不是把整句文字逐渐打出来。未读文字预先可见，用户可以浏览后文。

操作或事件	预期行为
播放语音	立即尝试播放，异步请求对齐数据
播放进行中	以播放器时钟计算已读前缀
暂停和继续	保留音频对象与当前位置，从暂停处继续
点击已对齐文字	跳到其声学单词的真实起点
未识别文字	可显示；没有可靠时间戳时不能精确跳播
字幕晚到	采样当前播放时间接上进度；流式已读部分不倒退
自然结束	当前已完成内容及末尾标点全部变为已读
中断或取消	保留已读部分，不能把未听内容标记为完成
气泡重新渲染	恢复已读状态和播放按钮，不重新播放音频

显示进度与跳播进度必须分开。真实词级时间戳允许在词内插值以平滑显示；点击词内任一字仍跳到那个词的真实起点。缺少时间戳时，展示层允许估算，但不能生成可点击的伪时间戳。

原实现没有独立的字幕横向滚动条，也没有每读一个字就强制滚动容器。聊天容器沿用原页面的滚动策略，用户浏览历史时不应被字幕拖回底部。

2 模块边界与数据契约

迁移时可拆成四层：播放器适配器提供时间与生命周期事件；对齐服务返回当前录音对应的时间戳；纯函数计算显示时间及文字映射；视图只维护按钮、颜色和状态。

TYPESCRIPT

```
type Cue = {
  text: string; // 一个 grapheme；必须与屏幕上显示的原文匹配
  start: number | null; // 单位：秒；null 表示无可靠定位
  end: number | null;
};

type Alignment = {
  text: string;
  audioIdentity: string; // 音频内容哈希或不可变版本 ID
  duration: number;
  source:
    | 'elevenlabs-forced-alignment'
    | 'deepgram-word-timestamps'
    | 'groq-word-timestamps';
  cues: Cue[];
};

// 以下为提取组件时建议实现的接口，不是现成产品 API。
interface AudioClock {
  currentTime: number;
  duration: number; // 优先采用解码后的真实时长
  paused: boolean;
  play(): Promise<void>;
  pause(): void;
  seek(seconds: number): void;
}
```

真实时间戳和显示时间分别存储。例如使用 `data-seek-time` 保存跳播位置、`data-reveal-time` 保存视觉推进时间、`.is-read` 标记已读，使用 `data-timing-status` 区分 `estimated` 与 `measured`；这些是接入示例命名。估算结果只写入展示状态。

绑定对齐结果前，检查文本、音频标识、时长、字符数，以及每个 grapheme 是否完全相等。音频重新合成或正文被编辑后，旧时间戳必须失效。流式片段和保存后的整段回放可能是分别合成的录音，即使文字相同也不能互用时间轴。

3 Unicode 分字和真实时间对齐

使用 `Intl.Segmenter(undefined, { granularity: 'grapheme' })` 分割文字，避免把组合音标或 emoji 拆坏。前端原实现没有 Segmenter 时退回 `Array.from`；该回退只能保证码点边界。后端纯对齐函数直接使用 Segmenter，接入端需保证运行环境支持。

对齐前通过 NFKC、转小写和保留 Unicode 字母数字生成匹配序列。显示仍保留原始字符和标点；不要把规范化后的文本覆盖回界面。

字符级强制对齐

`normalizeAlignment(text, result)` 验证供应商返回的 `characters`：起点不逆序、终点不早于起点、时间有限且不超过 3600 秒。首个起点不能小于零。规范化后的原文必须与供应商字符完全匹配，否则返回 `null`。标点继承前一个结束位置，起止相同。

词级识别回退

`alignRecognizedWords(text, words, { allowPartial })` 用最长公共子序列匹配原文与识别词。

- Deepgram 词置信度至少为 0.7；只保留完整匹配的供应商单词，不能从另一个单词里借几个字母。
- 默认匹配覆盖率至少为 80%。保存语音允许部分匹配时，最低为 35%，并要求至少 6 个连续匹配的规范化字母或汉字。
- 没匹配的 grapheme 返回 `{ start: null, end: null }`。
- 同一个识别词中的所有字共享该词的真实起止时间。
- 原文与识别序列长度乘积大于 4,000,000 时直接返回 `null`，限制动态规划内存和计算量。

Groq 的词结果没有逐词置信度；现实现为匹配流程补 `confidence: 1`。这只是输入适配，不能解释为供应商声称 100% 置信。它仍然经过完整词匹配与覆盖率门槛。

显示细分与跳播示例

```
JAVASCRIPT
// voiceRevealTimes 的完整实现见第 11 节。

const cues = [
  { text: '你', start: 2, end: 2.8 },
  { text: '好', start: 2, end: 2.8 },
  { text: '，', start: null, end: null },
  { text: '回', start: 5, end: 5.8 },
  { text: '来', start: 5, end: 5.8 },
];
const reveal = voiceRevealTimes(cues, 'deepgram-word-timestamps');
// [2, 2.4, null, 5, 5.4]
// 点击“好”仍 seek(2)；2.8 到 5 的停顿不能被摊平。
```

4 保存语音的播放与字幕更新

为每条语音维护播放器状态、对齐状态和已读字符数量。播放按钮触发音频加载及播放，同时异步请求字幕。不能 `await alignment` 后才 `play()`。

播放中通过 `requestAnimationFrame` 采样 `audio.currentTime`，并用 `timeupdate`、`seeked`、`playing`、`loadedmetadata` 校正。动画帧用于采样真实时钟，不能用固定间隔累加一个“假播放进度”。暂停、结束、卸载时取消帧循环。

```
JAVASCRIPT
function readCountAt(time, duration, cues, revealTimes, total) {
  if (!Number.isFinite(duration) || duration <= 0) return 0;
  time = Math.max(0, Math.min(duration, time));
  if (time >= duration - 0.02) return total;
```

JAVASCRIPT / CONTINUED

```
  const measured = cues.some((c) => Number.isFinite(c.start));
  if (!measured) return Math.floor((time / duration) * total); // 仅显示估算
  let last = -1;
  if (time > 0)
    cues.forEach((c, i) => {
      if (Number.isFinite(c.start) && revealTimes[i] <= time + 1e-5) last = i;
    });
  return last + 1;
}
```

实际更新时，仅切换新旧已读前缀之间的 `.is-read`，无需每帧改写全部文字。时间标签只在字符串变化时更新。标点和未匹配文字随下一处确认的已读前缀变深；自然结束后补齐末尾标点。

普通回放在完全没有测量时间时，按实际播放比例估算显示。存在部分测量数据后，按测量数据推进，未匹配区域不会凭空获得精确点击位置。普通回放的对齐数据晚到时会按真实时钟重新计算；“已读不回退”的额外保护主要用于流式通话，不应混为一谈。

点字跳播

点击先请求或等待这条录音的对齐结果。对齐失败给出明确提示；`start: null` 的字不可跳播。不要把 `null` 转成数字零。

对每次播放或点击增加请求版本号，异步加载完成后检查版本是否仍然有效，避免连续点击时旧结果覆盖新目标。DOM 被替换时重新查找当前气泡。

`audio.readyState >= 1` 时可设置 `currentTime`；否则在 `loadedmetadata` 后设置。以解码时长限制目标到 `[0, duration - 0.01]`。同一 Audio 对象还要有 seek 版本号，忽略旧的 metadata 回调。显式跳播需打标，防止错误恢复逻辑把播放重置到开头。

NEST 暂停后保留 Audio 对象供同一条语音续播，以减少 WebKit 对同一 Blob 反复构造播放器带来的问题。销毁或清缓存时释放不再使用的 Blob URL。

5 流式通话与迟到字幕

流式语音按 `generationId + seq` 归属队列。先向原生播放器或 Web Audio 播放通道提交音频，再异步为这一片段的相同字节请求字幕。字幕请求不能重新合成一段声音，也不能阻塞下一片段播放。

下面是独立字幕请求的通用示例，路径需由接入方定义：

HTTP

```
POST /api/captions/chunks
Content-Type: application/json

{
  "audio_data_url": "data:audio/mpeg;base64,...",
  "text": "本片段实际朗读内容",
  "duration": 8.2
}
```

返回 `{ "captions": [{ "text": "□", "start": 0.4, "end": 0.8 }] }` 或 `{ "captions": null }`。接口由宿主鉴权；示例省略了认证信息。服务端应限制文本长度、音频体积与时长，并验证实际媒体格式；时间单位为秒。示例使用 MPEG data URL，支持哪些 MIME 由接入服务明确约定。

收到响应后再次检查 generation 和片段归属。已取消或替换的片段丢弃结果。当前片段字幕到达时采样正在播放的时钟，不从零开始字幕动画。

片段绑定逻辑按片段序号把朗读内容匹配到对应气泡文本，并使用该片段的实际时长。可靠词时间区间保持不变；缺失区间在相邻测量区间之间或片段边界内估算显示。晚到的可靠时间可修正未读部分，已读前缀用最大值保留。

每个片段只在自然 `ended` 时完成全部文字及末尾标点。停止、打断和播放失败不能走同一完成分支。流式期间禁用文字跳播，避免误用整段回放的另一套时间轴。

临时消息 ID 在 `done` 或 `voice_ready` 后要映射到真实消息 ID；字幕状态和“已经播放”的归属一并转移。实际开始、取消或挂断后都应消费相应语音归属，避免迟到的完整录音被普通聊天自动播放再次触发。

6 后端对齐和缓存

保存回放可使用示例路由 `POST /api/captions/alignment`，提交当前可见文本和音频标识，由宿主后端校验访问权限并读取原录音。参考函数支持字符级强制对齐与词级时间戳两种输入；代码中的 ElevenLabs、Deepgram、Groq 来源标识用于识别输入格式，接入方自行选择适配器和回退顺序。

为每个对齐适配器设置超时与取消机制。音频播放独立于对齐回退链。服务商密钥只能在后端环境变量中，前端只调用自己的鉴权接口。

缓存键使用接入方定义的版本命名空间，加上音频内容与显示文字的联合哈希，例如 `SHA-256(audioDataURL + "\n" + visibleText)`。成功结果和空结果分别设置有效期；同键在途请求用 Map 合并。不能只用消息 ID 作为缓存依据。

前端缓存应设置容量上限、失败重试间隔和预热并发上限；服务端负缓存仍应限制无效上游请求。播放位置与流式已读状态可以保存在页面内存中，不代表跨设备听读历史。具体参数需按接入项目的音频长度、流量与内存预算确定。

保存语音 data URL 解析支持 `audio/webm;codecs=opus` 等 MIME 参数，不能用只支持简单 subtype 的正则。保留声音中的停顿、呼吸等真实间隔；清除用于控制 TTS 的标签时，只移除明确的已知标签，不误删正常括号正文。

7 长 MP3 在 WebKit 下的定位处理

分段 MP3 被字节拼接后，可能保留多个 `Info`、`Xing` 或 `VBRI` 索引。NEST 曾出现 Chromium 可跳到后段、WKWebView 却按首段时长解释整条语音的情况。因此 Chromium 通过不能替代 WebKit 验证。

参考函数 `normalizeVoiceMp3Seek(bytes)` 解析 MPEG Layer III 帧边界和合法索引位置；确认存在多个索引后，在本地副本中把相应四字节标记清零。帧长度、字节偏移与语音帧保持不变。单索引、无索引、损坏或截断数据直接返回原输入，不对任意字节做全局查找替换。

这个函数是对当前拼接方式的兼容处理，不是通用 MP3 修复器。非 MP3 不走该路径；新实现优先保证生产端输出可正确寻址的完整音频。原有录音无需重新合成或修改服务端存储。

8 宿主界面接入

本实现只开放字幕时序、文本对齐、已读状态和跳播逻辑。展示素材中的 NEST 气泡属于产品展示，不随算法开放其样式、主题参数或界面模板。

接入方提供自己的文本容器、播放控件、已读与未读视觉状态。字幕核心只输出字符、可定位时间、显示时间与已读前缀数量，不规定容器形状、配色、字体或布局。

保持自然段落，跳过不属于朗读正文的链接、代码和翻译。使用 `textContent` 创建文字节点，避免把外部文本注入 HTML。提供键盘操作与可理解的操作标签；长文需要处理逐字按钮造成的过多 Tab 停靠点。

9 迁移步骤与验收

1. 接入现有录音与 `AudioClock`，先完成播放、暂停、续播及时间采样。
2. 使用 grapheme 建立显示字符索引，再接入异步对齐结果。
3. 分开保存真实跳播时间和视觉细分时间，实现已读前缀更新。
4. 为点击加载和 metadata seek 添加版本校验，再处理 DOM 替换。
5. 若宿主有流式通话，独立建立片段时间轴与 generation 生命周期。
6. 接入宿主自己的视觉状态、焦点态及可访问标签；最后验证媒体格式兼容。

验收至少覆盖下列行为：

场景	验收依据
首次播放，对齐服务很慢或失败	音频仍先开始，显示可估算，无伪跳播时间
实际停顿、词内多字	停顿保留；词内显示细分，点击仍用词起点
混合语言、标点、组合音标和 emoji	文本完整，索引不漂移，无定位的字不跳到 0
连续点击两个不同位置	最后一次点击生效，旧请求不覆盖
暂停、续播、向前向后跳转	音频时间与已读前缀一致
已读时气泡被替换	保留播放器状态与对应文字状态
流式迟到字幕、取消后返回	已读不倒退，过期响应不复活播放
自然结束和用户打断	仅自然结束补齐当前片段尾部
长 MP3 在 Chromium 和 WebKit	靠后的目标保持在对应时间，不能返回开头
320px 到桌面、深浅主题、键盘	不溢出、不丢焦点提示、文字可读

第 12 节包含 11 项合成数据测试，可以与第 11 节核心代码一起运行。

测试覆盖算法提取，不包含真实供应商网络、iPhone 音频链路或完整产品端到端。接手 AI 必须使用目标宿主的实际音频补做浏览器和设备验收。

10 可直接交给 AI 的实现指令

AI IMPLEMENTATION PROMPT

请依据本文在当前项目实现语音逐字字幕逻辑。
先检查当前播放器、消息结构、主题变量和框架，再做最小集成。
复用本文第 11 节的纯函数；无需迁移整个 NEST 应用。

必须满足：
1. 音频先启动，对齐异步完成；不得重新合成录音来匹配字幕。
2. 使用真实播放时钟；grapheme 索引与可见文本完全一致。
3. seekTime 与 revealTime 分离。无测量时间时只估算显示，不允许精准跳播。
4. 保存回放和流式片段分别管理音频标识、时间轴与生命周期。
5. 处理暂停续播、过期响应、连续点击、DOM 替换、自然结束与中断。
6. 密钥放服务端；保持认证；不把示意时间轴用于真实录音。
7. 接入项目已有界面。不要推测或复刻展示素材中 NEST 气泡的样式。
8. 至少完成本文验收表；如暂时没有某条链路，明确交代范围。

请交付实现代码、运行方式、输入输出类型、可替换的对齐适配器和测试结果。
先说明已发现的宿主集成点，再直接完成实现；不要只返回概念方案。

11 核心算法实现

以下代码不依赖框架、网络或产品样式。AI 可按目标项目的模块组织方式直接复用；模块内部使用 Unicode grapheme 分割，需要运行环境支持 `Intl.Segmenter`。

JAVASCRIPT

```
// Reusable caption algorithms extracted from NEST for this reference package.
// Pure algorithms only. No credentials, recordings, conversations or backend calls.
const graphemes = (text) =>
  Array.from(
    new Intl.Segmenter(undefined, { granularity: 'grapheme' }).segment(text),
    (x) => x.segment,
  );
const letters = (text) =>
  Array.from(text.normalize('NFKC').toLowerCase()).filter((c) =>
    /[\p{L}\p{N}]/u.test(c),
  );
export function normalizeAlignment(text, result) {
  if (!Array.isArray(result?.characters) || !result.characters.length)
    return null;
  const spoken = [];
  let previous = 0;
  for (const item of result.characters) {
    if (
      typeof item.text !== 'string' ||
      !Number.isFinite(item.start) ||
      !Number.isFinite(item.end) ||
      item.start < previous ||
      item.end < item.start ||
      item.end > 3600
    )
      return null;
    previous = item.start;
    for (const char of letters(item.text))
      spoken.push({ char, start: item.start, end: item.end });
  }
  const units = graphemes(text);
  if (
    units.flatMap(letters).join('') !== spoken.map((x) => x.char).join('') ||
    !spoken.length
  )
    return null;
  let index = 0,
    lastEnd = spoken[0].start;
  return units.map((char) => {
    const count = letters(char).length;
    if (!count) return { text: char, start: lastEnd, end: lastEnd };
    const first = spoken[index],
      last = spoken[index + count - 1];
    index += count;
    lastEnd = last.end;
    return { text: char, start: first.start, end: last.end };
  });
}
// STT fallback is word-level: every character in a recognized word shares that
// word's real start. Unmatched text stays untimed; never interpolate over sounds.
export function alignRecognizedWords(
  text,
  words,
  { allowPartial = false } = {},
) {
  if (!Array.isArray(words)) return null;
```

JAVASCRIPT / CONTINUED

```
  const units = graphemes(text),
    input = units.flatMap((char, index) =>
      letters(char).map((c) => ({ char: c, index })),
    );
  const heard = [];
  let previous = 0;
  for (const [token, w] of words.entries()) {
    if (
      !Number.isFinite(w.start) ||
      !Number.isFinite(w.end) ||
      w.start < previous ||
      w.end < w.start ||
      w.end > 3600
    )
      return null;
    previous = w.start;
    if (!(w.confidence >= 0.7)) continue;
    for (const char of letters(w.word || ''))
      heard.push({ char, start: w.start, end: w.end, token });
  }
  if (!input.length || !heard.length || input.length * heard.length > 4000000)
    return null;
  const n = input.length,
    m = heard.length,
    cols = m + 1,
    dp = new Uint16Array((n + 1) * cols);
  for (let i = n - 1; i >= 0; i--)
    for (let k = m - 1; k >= 0; k--)
      dp[i * cols + k] =
        input[i].char === heard[k].char
          ? 1 + dp[(i + 1) * cols + k + 1]
          : Math.max(dp[(i + 1) * cols + k], dp[i * cols + k + 1]);
  const matches = new Map();
  let i = 0,
    k = 0;
  while (i < n && k < m) {
    if (input[i].char === heard[k].char) {
      matches.set(i, heard[k]);
      i++;
      k++;
    } else if (dp[(i + 1) * cols + k] >= dp[i * cols + k + 1]) i++;
    else k++;
  }
  // Keep only fully matched provider words; matching a letter inside another word is unsafe.
  const totals = new Map(),
    matched = new Map();
  for (const h of heard) totals.set(h.token, (totals.get(h.token) || 0) + 1);
  for (const h of matches.values())
    matched.set(h.token, (matched.get(h.token) || 0) + 1);
  for (const [index, h] of matches)
    if (totals.get(h.token) !== matched.get(h.token)) matches.delete(index);
  if (matches.size / n < 0.8) {
    // Saved mixed-language clips can contain spelling/pronunciation that STT
    // misses. Retain a substantial contiguous, measured phrase, not isolated
    // coincidental letters. Unmatched text remains explicitly untimed.
    let run = 0,
      longest = 0,
      previousInput = -2,
      previousHeard = -2;
    const heardIndex = new Map(heard.map((h, index) => [h, index]));
    for (const [index, h] of matches) {
      const hi = heardIndex.get(h);
      run =
```

JAVASCRIPT / CONTINUED

```
        index === previousInput + 1 && hi === previousHeard + 1 ? run + 1 : 1;
      longest = Math.max(longest, run);
      previousInput = index;
      previousHeard = hi;
    }
    if (!allowPartial || matches.size / n < 0.35 || longest < 6) return null;
  }
  const byUnit = new Map();
  for (const [index, h] of matches) {
    const u = input[index].index;
    if (!byUnit.has(u)) byUnit.set(u, h);
  }
  return units.map((char, index) => ({
    text: char,
    start: byUnit.get(index)?.start ?? null,
    end: byUnit.get(index)?.end ?? null,
  }));
}

export function voiceRevealTimes(timing, source) {
  const times = timing.map((t) => t.start);
  if (!['deepgram-word-timestamps', 'groq-word-timestamps'].includes(source))
    return times;
  for (let i = 0; i < timing.length;) {
    const item = timing[i];
    let end = i + 1;
    if (!Number.isFinite(item.start) || !(item.end > item.start)) {
      i++;
      continue;
    }
    while (
      end < timing.length &&
      timing[end].start === item.start &&
      timing[end].end === item.end
    )
      end++;
    for (let k = i; k < end; k++)
      times[k] = item.start + ((item.end - item.start) * (k - i)) / (end - i);
    i = end;
  }
  return times;
}
export function voiceTextUnits(text) {
  return (
    typeof Intl.Segmenter === 'function'
      ? Array.from(
          new Intl.Segmenter(undefined, { granularity: 'grapheme' }).segment(
            text,
          ),
          (s) => s.segment,
        )
      : Array.from(text)
  ).map((char) => ({ char }));
}

export function normalizeVoiceMp3Seek(bytes) {
  const markers = [];
  let offset = 0;
  const tag = (at, text) =>
    at + text.length <= bytes.length &&
    Array.from(text).every((c, i) => bytes[at + i] === c.charCodeAt(0));
```

JAVASCRIPT / CONTINUED

```
  while (offset < bytes.length) {
    if (tag(offset, 'ID3') && offset + 10 <= bytes.length) {
      const size =
        ((bytes[offset + 6] & 127) << 21) |
        ((bytes[offset + 7] & 127) << 14) |
        ((bytes[offset + 8] & 127) << 7) |
        (bytes[offset + 9] & 127);
      offset += 10 + size + (bytes[offset + 5] & 16 ? 10 : 0);
      continue;
    }
    if (offset + 128 === bytes.length && tag(offset, 'TAG')) break;
    if (
      offset + 4 > bytes.length ||
      bytes[offset] !== 255 ||
      (bytes[offset + 1] & 224) !== 224
    )
      return bytes;
    const version = (bytes[offset + 1] >> 3) & 3,
      layer = (bytes[offset + 1] >> 1) & 3;
    const rateIndex = bytes[offset + 2] >> 4,
      sampleIndex = (bytes[offset + 2] >> 2) & 3;
    if (
      version === 1 ||
      layer !== 1 ||
      rateIndex === 0 ||
      rateIndex === 15 ||
      sampleIndex === 3
    )
      return bytes;
    const rate =
      (version === 3
        ? [0, 32, 40, 48, 56, 64, 80, 96, 112, 128, 160, 192, 224, 256, 320]
        : [0, 8, 16, 24, 32, 40, 48, 56, 64, 80, 96, 112, 128, 144, 160])[
        rateIndex
      ] * 1000;
    const sampleRate =
      [44100, 48000, 32000][sampleIndex] /
      (version === 3 ? 1 : version === 2 ? 2 : 4);
    const length =
      Math.floor(((version === 3 ? 144 : 72) * rate) / sampleRate) +
      ((bytes[offset + 2] >> 1) & 1);
    if (offset + length > bytes.length) return bytes;
    const mono = bytes[offset + 3] >> 6 === 3;
    const sideInfo = version === 3 ? (mono ? 17 : 32) : mono ? 9 : 17;
    const xing = offset + 4 + (bytes[offset + 1] & 1 ? 0 : 2) + sideInfo;
    if (
      xing + 4 <= offset + length &&
      (tag(xing, 'Info') || tag(xing, 'Xing'))
    )
      markers.push(xing);
    const vbri = offset + 36;
    if (vbri + 4 <= offset + length && tag(vbri, 'VBRI')) markers.push(vbri);
    offset += length;
  }
  if (markers.length < 2) return bytes;
  const normalized = bytes.slice();
  for (const at of markers) normalized.fill(0, at, at + 4);
  return normalized;
}
```

12 合成数据自检

以下测试只使用合成文本和合成 MP3 帧。需要运行时，把第 11 节代码与下方测试顺序放入同一个临时模块 `caption-check.mjs`，执行 `node --test caption-check.mjs` 即可；无需下载其他代码文件。测试部分使用 Node.js 内置模块，不应放入浏览器运行包。

JAVASCRIPT

```
import test from 'node:test';
import assert from 'node:assert/strict';

test('graphemes preserve a combined accent and a family emoji', () => {
  assert.deepEqual(
    voiceTextUnits(
      'e\u0301\u{1f468}\u200d\u{1f469}\u200d\u{1f467}\u200d\u{1f466}',
    ),
    [
      { char: 'e\u0301' },
      { char: '\u{1f468}\u200d\u{1f469}\u200d\u{1f467}\u200d\u{1f466}' },
    ],
  );
});
test('forced alignment preserves introductory and internal audio gaps', () => {
  const cues = normalizeAlignment('你好，回来。', {
    characters: [
      { text: '你', start: 2, end: 2.4 },
      { text: '好', start: 2.4, end: 2.8 },
      { text: '回', start: 5, end: 5.4 },
      { text: '来', start: 5.4, end: 5.8 },
    ],
  });
  assert.deepEqual(
    cues.map((c) => c.start),
    [2, 2.4, 2.8, 5, 5.4, 5.8],
  );
});
test('mismatched or reversed alignment is rejected', () => {
  assert.equal(
    normalizeAlignment('你好', {
      characters: [{ text: '你错', start: 1, end: 2 }],
    }),
    null,
  );
  assert.equal(
    normalizeAlignment('你好', {
      characters: [
        { text: '你', start: 2, end: 3 },
        { text: '好', start: 1, end: 2 },
      ],
    }),
    null,
  );
  assert.equal(normalizeAlignment('你好', {}), null);
});
test('word timing remains measured while visual reveal subdivides the word', () => {
  const cues = alignRecognizedWords('你好，回来。', [
    { word: '你好', start: 2, end: 2.8, confidence: 0.99 },
    { word: '回来', start: 5, end: 5.8, confidence: 0.99 },
  ]);
  assert.deepEqual(
    cues.map((c) => c.start),
    [2, 2, null, 5, 5, null],
  );
```

JAVASCRIPT / CONTINUED

```
  assert.deepEqual(voiceRevealTimes(cues, 'deepgram-word-timestamps'), [
    2,
    2.4,
    null,
    5,
    5.4,
    null,
  ]);
  assert.deepEqual(
    cues.map((c) => c.start),
    [2, 2, null, 5, 5, null],
  );
  assert.deepEqual(voiceRevealTimes(cues, 'elevenlabs-forced-alignment'), [
    2,
    2,
    null,
    5,
    5,
    null,
  ]);
});
test('low-confidence recognition does not produce seek times', () => {
  assert.equal(
    alignRecognizedWords('你好', [
      { word: '你好', start: 0, end: 1, confidence: 0.3 },
    ]),
    null,
  );
});
test('saved partial alignment keeps an anchored phrase and leaves the rest untimed', () => {
  const text = '这是一段测试文字example未对齐部分';
  const words = [
    { word: '这是一段测试文字', start: 0.2, end: 2, confidence: 0.99 },
  ];
  assert.equal(alignRecognizedWords(text, words), null);
  const cues = alignRecognizedWords(text, words, { allowPartial: true });
  assert.equal(cues[0].start, 0.2);
  assert.equal(cues.at(-1).start, null);
});
test('sparse coincidental matches do not satisfy partial alignment', () => {
  const words = ['甲', '丙', '戊', '庚'].map((word, i) => ({
    word,
    start: i * 2,
    end: i * 2 + 1,
    confidence: 0.99,
  }));
  assert.equal(
    alignRecognizedWords('甲乙丙丁戊己庚辛壬癸', words, {
      allowPartial: true,
    }),
    null,
  );
});
function frame(marker) {
  const b = new Uint8Array(576);
  b.set([255, 251, 152, 192]);
  if (marker) b.set(Buffer.from(marker), 21);
  return b;
}
const join = (...parts) =>
  new Uint8Array(Buffer.concat(parts.map((x) => Buffer.from(x))));
```

JAVASCRIPT / CONTINUED

```
test('multiple MP3 indexes are disabled without moving or changing speech frames', () => {
  const bytes = join(
    frame('Info'),
    frame(),
    frame('Info'),
    frame(),
    frame('Xing'),
    frame(),
  );
  const fixed = normalizeVoiceMp3Seek(bytes),
    expected = bytes.slice();
  for (const at of [21, 1173, 2325]) expected.fill(0, at, at + 4);
  assert.notEqual(fixed, bytes);
  assert.deepEqual(fixed, expected);
  assert.equal(Buffer.from(bytes.slice(21, 25)).toString(), 'Info');
});
test('single MP3 indexes and marker-like speech bytes remain unchanged', () => {
  const bytes = join(frame('Info'), frame());
  bytes.set(Buffer.from('Info'), 650);
  assert.equal(normalizeVoiceMp3Seek(bytes), bytes);
});
test('malformed and truncated MP3 data remain unchanged', () => {
  const bytes = join(frame('Info'), frame('Info'), new Uint8Array([1, 2, 3]));
  assert.equal(normalizeVoiceMp3Seek(bytes), bytes);
  const fake = new Uint8Array(Buffer.from('Info fake Info'));
  assert.equal(normalizeVoiceMp3Seek(fake), fake);
});
test('ID3-prefixed segments keep tag bytes and offsets intact', () => {
  const id3 = new Uint8Array([73, 68, 51, 4, 0, 0, 0, 0, 0, 0]);
  const bytes = join(
    id3,
    frame('Info'),
    frame(),
    id3,
    frame('Xing'),
    frame(),
  );
  const fixed = normalizeVoiceMp3Seek(bytes);
  assert.notEqual(fixed, bytes);
  assert.deepEqual(fixed.slice(0, 10), id3);
  assert.deepEqual(Array.from(fixed.slice(31, 35)), [0, 0, 0, 0]);
});
```